

Adaptive GPU Optimization for Deep Learning Workloads Using Evolutionary Algorithms

Sameer Rajendra

Department of Electrical and Computer Engineering
Stevens Institute of Technology
Hoboken, NJ, USA
srajendr1@stevens.edu

Prof. Hong Man

Department of Electrical and Computer Engineering
Stevens Institute of Technology
Hoboken, NJ, USA
hman@stevens.edu

Abstract—Modern GPU deployments for deep learning suffer from inefficient resource utilization due to default “one-size-fits-all” parameter configurations, leading to wasted energy and suboptimal performance. The challenge lies in simultaneously optimizing multiple hardware parameters (power limits, memory clocks) and software parameters (batch size, precision modes, kernel implementations) while maintaining model accuracy and ensuring hardware safety. This work presents an Adaptive Evolutionary GPU Optimization Framework (AEGOF) that employs NSGA-II multi-objective evolutionary algorithm with hybrid safety heuristics to dynamically tune 4-8 GPU parameters. Experimental evaluation across 238 configurations demonstrates substantial improvements: ResNet50 training discovers two Pareto-optimal solutions achieving $1.65\times$ (Best Performance) and $2.68\times$ (Energy Efficient) efficiency gains in accuracy per Watt-hour, while Llama 2 inference delivers $4.07\times$ (Best Performance) and $8.09\times$ (Energy Efficient) efficiency multipliers in tokens per Watt-hour, substantially exceeding the proposed 15-20% improvement target.

Index Terms—GPU optimization, evolutionary algorithms, deep learning, power efficiency, NSGA-II, Pareto optimization

I. INTRODUCTION

Artificial Intelligence’s computational demands consume vast amounts of electrical power, with GPUs drawing hundreds of watts during training and inference operations. Modern datacenter GPUs such as the NVIDIA H100 Tensor Core can consume up to 700W under peak load [8], contributing significantly to operational costs and environmental impact. The research problem addressed in this work is the suboptimal utilization of GPU resources caused by default parameter configurations that fail to adapt to specific workload characteristics.

A. Background and Importance

GPUs expose numerous tunable parameters including power limits, memory clock frequencies, batch sizes, numerical precision modes, and kernel implementations. These parameters significantly impact both performance and energy efficiency [1]. However, default “one-size-fits-all” settings are rarely optimal for specific computational patterns, leading to wasted energy and underutilized hardware capacity. Manually finding optimal configurations is infeasible due to the high-dimensional parameter space and complex interactions between settings.

The importance of this problem is threefold. First, energy efficiency directly translates to operational costs in cloud deployments where thousands of GPUs operate continuously. Second, environmental sustainability requires reducing AI’s carbon footprint [9]. Third, faster training and inference enable more rapid research iteration and better user experiences in production systems.

B. Challenges

Several challenges complicate GPU parameter optimization:

- 1) **High-dimensional parameter space:** Modern GPUs have 4-8+ tunable parameters with both discrete (precision modes, kernel types) and continuous (power limits, clock speeds) variables, creating exponentially large search spaces.
- 2) **Multi-objective optimization:** Training workloads require balancing accuracy, speed, and energy consumption objectives that often conflict. Inference workloads must maximize throughput while minimizing power draw.
- 3) **Hardware safety constraints:** Configurations must respect thermal limits, power budgets, and stability requirements to prevent hardware damage [12].
- 4) **Expensive evaluation:** Each configuration requires empirical measurement through actual workload execution, limiting the number of evaluations possible.
- 5) **Model accuracy preservation:** Optimization must maintain or improve model quality, not sacrifice accuracy for speed.

C. Related Work and Limitations

Existing GPU tuning approaches fall into three categories. Reinforcement learning methods like DRLCAP and EnergyUCB optimize frequency and voltage scaling but typically address only 1-3 parameters and rarely incorporate accuracy constraints [1]. Static heuristic approaches use fixed rules that cannot adapt to diverse workload characteristics [12]. Grid search and manual tuning become infeasible with more than 3-4 parameters due to exponential search space growth [11].

These approaches ignore critical interactions between hardware and software parameters, struggle to scale with more tuning variables, and lack robust constraint handling for hardware safety. Evolutionary algorithms have shown promise

in similar hardware optimization domains [5] but have not been comprehensively applied to GPU parameter tuning with explicit accuracy preservation and safety constraints.

D. Main Contributions

This work makes the following novel contributions:

- 1) **Hybrid Evolutionary Framework:** Development of AEGOF combining NSGA-II multi-objective evolutionary algorithm [2] with domain-informed safety heuristics for simultaneous optimization of 4-8 GPU parameters across training and inference workloads.
- 2) **Pareto-Optimal Solution Discovery:** Demonstration of distinct optimization regimes for both training and inference performance-focused vs. energy-focused—enabling deployment flexibility.
- 3) **Comprehensive Experimental Validation:** Evaluation across 238 configurations on NVIDIA H100 GPUs with mathematical proofs of efficiency improvements for ResNet50 training and Llama 2 inference.
- 4) **Safety-Aware Optimization:** Integration of proactive constraint handling preventing thermal violations and configuration failures while maintaining aggressive exploration.

E. Paper Organization

The remainder of this paper is organized as follows. Section II formally defines the GPU parameter optimization problem and system model. Section III presents the Adaptive Evolutionary GPU Optimization Framework including evolutionary operators and hybrid heuristics. Section IV provides numerical results and analysis for both training and inference workloads. Section V concludes with key findings and future directions.

II. PROBLEM STATEMENT

This section formally defines the GPU parameter optimization problem across two representative workload types: supervised deep learning training (Phase 1) and large language model inference (Phase 2).

A. System Context

The target hardware platform is the NVIDIA H100 PCIe GPU [8], a datacenter-class accelerator with the following specifications:

- 80 GB HBM3 memory with 3.35 TB/s bandwidth
- 4th-generation Tensor Cores supporting FP8/FP16/BF16/FP32 precision
- Configurable 350-700W thermal design power (TDP)
- 16,896 CUDA cores across 132 streaming multiprocessors
- Memory clock adjustment range: 0-400 MHz offset
- CUDA 12.2 toolkit with PyTorch 2.3 integration

Phase 1 Workload: ResNet50 convolutional neural network training on CIFAR-10 dataset (60,000 images, 10 classes) measuring test accuracy $A(x)$ (%), training time per epoch $T(x)$ (seconds), and power consumption $P(x)$ (Watts).

Phase 2 Workload: Llama 2 (7B parameters) large language model text generation inference measuring throughput $\Phi(x)$ (tokens/second) and power consumption $P(x)$ (Watts) across varied sequence lengths and batch sizes.

B. Configuration Parameter Space

For Phase 1 (Training), each configuration vector is:

$$x_{\text{train}} = [b, p, m, r] \quad (1)$$

where the parameters are defined as:

- $b \in \{32, 64, 128\}$ is batch size
- $p \in [200, 300, 400, 500, 600, 700]$ is power limit (Watts)
- $m \in [0, 300]$ is memory clock offset (MHz)
- $r \in \{\text{FP16}, \text{FP32}\}$ is numerical precision mode

For Phase 2 (Inference), the configuration space expands to 8 parameters:

$$x_{\text{infer}} = [b, s, p, \text{prec}, \text{bs}, \text{tc}, \text{fk}, \text{ak}] \quad (2)$$

where the parameters are defined as:

- $b \in \{1, 2, 4, 8, 16, 32\}$ is batch size
- $s \in \{128, 256, 512, 1024\}$ is sequence length
- $p \in [300, 500, 600, 700]$ is power limit (Watts)
- $\text{prec} \in \{\text{FP16}, \text{BF16}\}$ is precision mode
- $\text{bs} \in \{64, 128, 256\}$ is transformer block size
- $\text{tc} \in \{\text{tile_16x16}, \text{tile_32x32}\}$ is tile configuration
- $\text{fk} \in \{\text{standard}, \text{fused_mha_ffn}\}$ is kernel fusion strategy
- $\text{ak} \in \{\text{math}, \text{cutlass}, \text{mem_efficient}\}$ is attention kernel implementation

C. Optimization Objectives

Phase 1 Multi-Objective Formulation:

$$\text{maximize } A(x) \quad (\text{test accuracy}) \quad (3)$$

$$\text{minimize } T(x) \quad (\text{training time}) \quad (4)$$

$$\text{maximize } \eta_{\text{train}}(x) = \frac{A(x)}{E(x)} \quad (\text{accuracy per energy}) \quad (5)$$

where energy consumption $E(x) = \frac{P(x) \cdot T(x)}{3600}$ is measured in Watt-hours (Wh).

Phase 2 Multi-Objective Formulation:

$$\text{maximize } \Phi(x) \quad (\text{throughput}) \quad (6)$$

$$\text{maximize } \eta_{\text{infer}}(x) = \frac{\Phi(x) \cdot 3600}{P(x)} \quad (\text{tokens per Wh}) \quad (7)$$

The use of Watt-hours (Wh) as the energy metric enables direct comparison with electricity billing units and facilitates datacenter cost analysis.

D. Constraints

Hard constraints enforced during optimization:

$$\Theta(x) \leq 85^\circ\text{C} \quad (\text{thermal safety}) \quad (8)$$

$$P(x) \leq 700 \text{ W} \quad (\text{power budget}) \quad (9)$$

$$\text{Error}(x) = \text{None} \quad (\text{runtime stability}) \quad (10)$$

$$A(x) \geq 0.98 \cdot A_{\text{baseline}} \quad (\text{accuracy preservation, Phase 1}) \quad (11)$$

These constraints ensure hardware safety and prevent model quality degradation during optimization.

E. Baseline Reference Configurations

Phase 1 Baseline: Default settings achieve test accuracy $A_{\text{baseline}} = 81.75\%$ with training time $T_{\text{baseline}} = 31.45$ seconds per epoch at power $P_{\text{baseline}} = 700\text{W}$, consuming $E_{\text{baseline}} = 6.12$ Wh per epoch.

Phase 2 Baseline: Unoptimized configuration (standard kernels, mem_efficient attention, long sequences) achieves throughput $\Phi_{\text{baseline}} = 12,306$ tokens/second at power $P_{\text{baseline}} = 700\text{W}$, delivering 63,288 tokens/Wh.

III. ADAPTIVE EVOLUTIONARY GPU OPTIMIZATION FRAMEWORK

This section presents the Adaptive Evolutionary GPU Optimization Framework (AEGOF), the core contribution of this work. AEGOF employs a hybrid approach combining NSGA-II multi-objective evolutionary algorithm [2] with domain-informed safety heuristics to efficiently navigate high-dimensional GPU parameter spaces while ensuring hardware safety and model accuracy preservation.

A. Framework Architecture

AEGOF consists of two primary components:

1. Evolutionary Algorithm Core: The framework implements NSGA-II [2], a proven multi-objective evolutionary algorithm chosen for its ability to discover Pareto-optimal solution sets balancing conflicting objectives (accuracy vs. speed, throughput vs. power). NSGA-II naturally handles discrete-continuous mixed-variable spaces and requires no gradient information, making it suitable for black-box GPU optimization.

2. Hybrid Heuristics Layer: Domain-informed constraints provide three critical functions: (a) guided initialization biasing the initial population toward safe, promising regions based on GPU architecture knowledge, (b) constrained mutation ensuring genetic operations remain within feasible bounds, and (c) proactive safety checking preventing thermal violations and configuration failures before hardware damage occurs.

This hybrid design addresses the key challenge of balancing exploration (discovering novel high-performance configurations) with exploitation (refining known good solutions) while maintaining hardware safety throughout the optimization process.

B. Population-Based Search

The evolutionary search maintains a population of $N = 30$ candidate configurations, providing sufficient diversity while remaining computationally tractable given the expensive fitness evaluation (20-40 seconds per training configuration, 5-10 seconds per inference configuration).

Initialization Strategy: The initial population P_0 combines two generation methods:

- 50% Latin Hypercube Sampling: Ensures broad coverage of the parameter space through stratified sampling, reducing the risk of missing promising regions.
- 50% Heuristic Seeding: Biases initialization toward empirically promising regions based on GPU architecture

knowledge moderate power (500-600W avoiding thermal throttling), reduced precision (FP16/BF16 for Tensor Core utilization [7]), larger batch sizes for improved GPU occupancy.

This hybrid initialization accelerates convergence by 30-40% compared to pure random initialization while maintaining population diversity for effective exploration [5].

C. Evolutionary Operators

Parent Selection: Tournament selection with tournament size 3 chooses parents based on two criteria applied hierarchically [2]: (1) Pareto rank (lower is better) solutions on the first Pareto front dominate all others, and (2) crowding distance (higher is better) maintains diversity by preferring less crowded regions.

Crossover: Uniform crossover with probability $p_c = 0.7$ creates offspring by randomly selecting each parameter from either parent with equal probability.

Mutation: Parameter-type-specific mutation operators adapt to variable characteristics. For continuous parameters (power limit p , memory clock offset m), Gaussian perturbation:

$$x'_i = x_i + \mathcal{N}(0, \sigma_i^2) \quad \text{where} \quad \sigma_i = 0.1 \cdot (\max_i - \min_i) \quad (12)$$

with boundary reflection to maintain feasibility. For categorical parameters (r , prec, fk, ak, tc):

$$x'_i \sim \text{Uniform}(\text{domain}_i \setminus \{x_i\}) \quad (13)$$

where x_i is the current value and $\text{domain}_i \setminus \{x_i\}$ represents all possible values excluding the current value. For discrete numeric parameters (batch size b , sequence length s), neighbor selection (60%) or uniform sampling (40%). Base mutation probability $p_m = 0.15$ per gene decreases by 20% every 2 generations.

D. Multi-Objective Fitness Evaluation

Each candidate undergoes empirical evaluation through actual workload execution. The fitness vector construction differs by phase:

$$\mathbf{f}(x) = \begin{cases} [A(x), -T(x), \eta_{\text{train}}(x)] & \text{Phase 1} \\ [\Phi(x), \eta_{\text{infer}}(x)] & \text{Phase 2} \end{cases} \quad (14)$$

Following fitness evaluation, non-dominated sorting [2] ranks the population into Pareto fronts where solutions in F_1 are non-dominated by any other solution.

E. Hybrid Heuristics for Safety and Convergence

The heuristics layer implements proactive constraint handling through penalty-based fitness adjustment:

$$f'(x) = \begin{cases} f(x) & \text{if feasible}(x) \\ -10^6 & \text{otherwise} \end{cases} \quad (15)$$

where $\text{feasible}(x)$ verifies:

- Temperature $\Theta(x) \leq 85^\circ\text{C}$ (thermal safety)
- Power $P(x) \leq 700\text{W}$ (budget compliance)
- Runtime stability (no crashes or errors)
- Accuracy preservation $A(x) \geq 0.98 \cdot A_{\text{baseline}}$ (Phase 1)

F. Termination and Convergence

The algorithm terminates when either fixed generation limit is reached ($G = 6$ for Phase 1, $G = 8$ for Phase 2) or fitness plateau is detected (no improvement in hypervolume for 3 consecutive generations). Crowding distance mechanisms maintain 8-12 non-dominated solutions per generation.

G. Computational Complexity

Per-generation complexity is $O(MN^2)$ where M is the number of objectives (2-3) and N is population size (30), dominated by non-dominated sorting. Total optimization time is 60-90 minutes per workload (6-8 generations $\times$ 30 individuals $\times$ 20-40s evaluation time), acceptable for establishing reusable configuration libraries.

H. Rationale for Design Choices

The framework design addresses specific challenges:

- 1) **Why NSGA-II?** Multi-objective optimization naturally captures performance-energy trade-offs without manual weight tuning. Pareto fronts provide deployment flexibility [2].
- 2) **Why Hybrid Heuristics?** Pure evolutionary approaches often violate safety constraints. Domain knowledge accelerates convergence while preventing hardware damage [1].
- 3) **Why Parameter-Specific Mutation?** Uniform mutation performs poorly on mixed-variable spaces. Tailored operators respect variable semantics (e.g., nearby batch sizes vs. unrelated precision modes) [5].
- 4) **Why Population Size 30?** Balances diversity (covering parameter space) against evaluation cost. Empirically determined as minimum for reliable convergence across both workloads.

IV. NUMERICAL RESULTS AND ANALYSIS

This section presents comprehensive experimental results demonstrating AEGOF's effectiveness across ResNet50 training (Phase 1) and Llama 2 inference (Phase 2) workloads. All experiments execute on NVIDIA H100 PCIe GPU with CUDA 12.2 and PyTorch 2.3 [8].

A. Phase 1: ResNet50 Training Optimization

Experimental Setup: ResNet50 architecture adapted for CIFAR-10 (50,000 training images, 10,000 test images, 32×32 resolution, 10 classes). Training protocol uses SGD optimizer with momentum 0.9, learning rate 0.001, and cross-entropy loss. Each configuration undergoes single-epoch evaluation to measure test accuracy and training time. NVML library monitors power consumption and GPU temperature throughout execution.

Search Statistics: AEGOF evaluated 113 configurations across 6 evolutionary generations with $> 95\%$ success rate. The algorithm discovered optimal configurations by generation 4, with subsequent generations refining power and memory clock settings.

Three-Way Configuration Comparison: Table I presents baseline, performance-optimized, and energy-optimized configurations spanning the discovered Pareto frontier.

Baseline Configuration: Employs default GPU settings with small batch size 32, full precision (FP32), 700W power, and no memory clock offset. This configuration achieves only 81.75% test accuracy with 31.45 second training time, consuming 6.12 Wh per epoch. The small batch size results in poor GPU utilization and delivers only 13.37%/Wh efficiency.

Best Performance Configuration: Optimizes for maximum accuracy and speed, achieving 90.21% test accuracy (8.46 percentage point improvement) with 20.99 second training time ($1.50 \times$ speedup) at 700W power. Energy consumption decreases to 4.08 Wh per epoch despite maintaining high power, resulting in 22.10%/Wh efficiency. Key optimizations include batch size 128 ($4 \times$ baseline) improving gradient stability and GPU arithmetic intensity, FP16 precision leveraging Tensor Cores for $2 \times$ throughput [7], and +300 MHz memory clock offset reducing memory bottlenecks. This configuration delivers $1.65 \times$ efficiency multiplier, suitable for performance-critical training.

Energy Efficient Configuration: Achieves 99.7% of Best Performance accuracy (89.96% vs. 90.21%) while consuming only 61.5% of the energy (2.51 Wh vs. 4.08 Wh). Critical design choices include 400W power limit (43% reduction vs. baseline) exploiting the power-performance saturation region, batch size 128 and FP16 precision matching Best Performance for computational efficiency, and +100 MHz memory clock offset balancing performance with stability. Energy consumption of 2.51 Wh per epoch yields 35.78%/Wh efficiency, delivering $2.68 \times$ efficiency multiplier the highest for training making it optimal for energy-constrained environments.

Efficiency Gains:

Best Performance achieves $1.65 \times$ efficiency than baseline configuration.

Proof: Define efficiency as $\eta_{\text{train}}(x) = A(x)/E(x)$ where $E(x)$ is energy in Wh.

Baseline: $\eta_{\text{base}} = 81.75/6.12 = 13.37 \text{ \%/Wh}$

Best Performance: $\eta_{\text{perf}} = 90.21/4.08 = 22.10 \text{ \%/Wh}$

Efficiency Gain: $22.10/13.37 = 1.65 \times$

Energy Efficient configuration achieves $2.68 \times$ efficiency gain compared to baseline configuration.

Proof: Energy Efficient: $\eta_{\text{eff}} = 89.96/2.51 = 35.78 \text{ \%/Wh}$

Efficiency gain: $35.78/13.37 = 2.68 \times$

Parameter Interaction Analysis:

Batch size increase from 32 to 128 yields 3-8 percentage point accuracy improvements and 25-35% speedup. FP16 precision achieves comparable or superior accuracy to FP32 while completing 30-40% faster through $2 \times$ Tensor Core throughput [7]. Power saturation above 500W enables Energy Efficient's 400W configuration to achieve 98.9% of Best Performance speed with 38.5% lower energy consumption per epoch. Memory clock offsets contribute 1-3% marginal improvements, most effective when combined with large batches in memory-bound phases.

TABLE I
PHASE 1: RESNET50 TRAINING THREE-WAY CONFIGURATION COMPARISON

Parameter/Metric	Baseline	Best Performance	Energy Efficient
<i>Hardware Parameters</i>			
Power Limit (W)	700	700	400
Memory Clock Offset (MHz)	+0	+300	+100
<i>Software Parameters</i>			
Batch Size	32	128	128
Precision	FP32	FP16	FP16
<i>Performance Metrics</i>			
Test Accuracy (%)	81.75	90.21	89.96
Training Time (s/epoch)	31.45	20.99	22.63
Energy per Epoch (Wh)	6.12	4.08	2.51
Accuracy per Wh (%/Wh)	13.37	22.10	35.78
<i>Improvements vs. Baseline</i>			
Accuracy Gain	—	+8.46 pts	+8.21 pts
Speedup	—	1.50×	1.39×
Energy Reduction	—	33.3%	59.0%
Efficiency Multiplier	—	1.65×	2.68×

B. Phase 2: Llama 2 Inference Optimization

Experimental Setup: Llama 2 7B parameter model (32 transformer layers, 4096 hidden dimension, 32 attention heads) performing text generation inference. The framework evaluates specialized kernel implementations including CUTLASS [4] and Flash Attention for optimized attention computation on H100 architecture.

Evaluation Protocol: 20-step inference benchmark with 3-step warmup. Timeout: 60s (all successful runs ≤ 15 s).

Measurement Methodology: NVML at **10 Hz (10 samples/second)** captures transient power spikes. MFU calculation per H100 specs (989 TFLOPS BF16, 1,979 TFLOPS FP16).

Success/Failure Analysis: 125 attempted, 82 successful (65.643 failed (34.4(2) GPU/kernel failures: 0 (0 from software integration issues, NOT hardware limitations. Random distribution across parameter space—no clustering or bias.

Variance Analysis: Throughput std dev: 8,430 tokens/s (range: 12,306–42,879), demonstrating high sensitivity to parameters.

Search Statistics: AEGOF evaluated 125 configurations across 8 evolutionary generations. Success rate: 82 configurations (65.6%) completed successfully, while 43 (34.4%) failed due to memory errors or kernel incompatibilities. Optimal configurations emerged in generations 6–7.

Three-Way Configuration Comparison: Table II presents baseline, performance-optimized, and energy-optimized configurations spanning the discovered Pareto frontier.

Baseline Configuration: Represents typical unoptimized deployment using standard (non-fused) kernels, mem_efficient attention (slowest but most memory-conservative option), and long sequences (512 tokens). This configuration achieves only 12,306 tokens/second at 700W, delivering 63,288 tokens/Wh and consuming 15.80 mWh per token due to suboptimal GPU utilization.

Best Performance Configuration: Optimizes for maximum absolute prefill throughput, achieving 42,879 tokens/second

at 600W. Energy efficiency improves to 257,274 tokens/Wh, consuming only 3.89 mWh per token. Key optimizations include BF16 precision for numerical stability in attention [7], small batch size 4 for latency-sensitive serving, moderate sequence length 256 balancing throughput and memory, fused kernels (fused_mha_ffn) combining multi-head attention and feed-forward network computation to reduce memory traffic [4], and CUTLASS attention kernels optimized for H100 Tensor Cores. This configuration delivers 3.48× throughput improvement and 4.07× efficiency gain.

Energy Efficient Configuration: Achieves 99.6% of Best Performance throughput (42,688 tokens/second) while consuming only 50% of the power (300W vs. 600W). Energy efficiency reaches 512,256 tokens/Wh with only 1.95 mWh per token. Critical design choices include FP16 precision, batch size 8, shortest sequence length 128 minimizing quadratic attention complexity, fused kernels and CUTLASS attention matching Best Performance, and 300W power limit exploiting the performance-power saturation region. This configuration delivers 8.09× efficiency multiplier the highest across all experiments making it optimal for cost-conscious or carbon-constrained datacenter deployments [9].

Efficiency Gains:

Best Performance configuration achieves 4.07x efficiency gain compared to baseline configuration w.r.t tokens per Wh.

Proof: Define efficiency as $\eta_{\text{infer}}(x) = \Phi(x) \times 3600/P(x)$ where $\Phi(x)$ is throughput (tokens/s) and $P(x)$ is power (W).

Baseline: $\eta_{\text{base}} = 12,306 \times 3600/700 = 63,288$ tokens/Wh

Best Performance: $\eta_{\text{perf}} = 42,879 \times 3600/600 = 257,274$ tokens/Wh

Gain: $257,274/63,288 = 4.07x$

Energy Efficient configuration achieves 8.09x efficiency gain compared to baseline configuration.

Proof: Energy Efficient: $\eta_{\text{eff}} = 42,688 \times 3600/300 = 512,256$ tokens/Wh

Gain: $512,256/63,288 = 8.09x$

TABLE II
PHASE 2: LLAMA 2 INFERENCE THREE-WAY CONFIGURATION COMPARISON

Parameter/Metric	Baseline	Best Performance	Energy Efficient
<i>Hardware Parameters</i>			
Power Limit (W)	700	600	300
<i>Software Parameters</i>			
Precision	FP16	BF16	FP16
Batch Size	8	4	8
Sequence Length	512	256	128
Block Size	256	128	256
Tile Configuration	32×32	16×16	32×32
Fused Kernel	standard	fused_mha_ffn	fused_mha_ffn
Attention Kernel	mem_efficient	cutlass	cutlass
<i>Performance Metrics</i>			
Throughput (tokens/s)	12,306	42,879	42,688
Power (W)	700	600	300
Tokens per Wh	63,288	257,274	512,256
Energy per Token (mWh)	15.80	3.89	1.95
<i>Improvements vs. Baseline</i>			
Throughput Multiplier	–	3.48×	3.47×
Efficiency Multiplier	–	4.07×	8.09×
Energy/Token Reduction	–	75.4%	87.7%

Sustained Inference Analysis: For 1 hour continuous inference at equal energy budget, Energy Efficient generates $8.09\times$ more tokens than baseline.

Proof: At energy budget $E = 700$ Wh (baseline 1-hour consumption):

Baseline produces: $63,288 \text{ tokens/Wh} \times 700 \text{ Wh} = 44,301,600 \text{ tokens}$

Energy Efficient produces: $512,256 \text{ tokens/Wh} \times 700 \text{ Wh} = 358,579,200 \text{ tokens}$

Gain: $358,579,200 / 44,301,600 = 8.09x$

Parameter Interaction Analysis:

Fused kernels (fused_mha_ffn) appear in both optimized configurations, providing 25-30% throughput gains by reducing memory traffic and kernel launch overhead [4]. CUTLASS kernels deliver $2\text{-}3\times$ speedup over mem_efficient attention through H100-specific optimizations. Throughput peaks at sequence length 128 (512,256 tokens/Wh), declining at 256 and severely degrading at 512 due to $O(n^2)$ attention complexity. BF16 provides numerical stability advantages for attention computation while both BF16 and FP16 substantially outperform FP32 through Tensor Core utilization [7].

C. Cross-Phase Comparative Analysis

Table III summarizes efficiency gains and Pareto frontier characteristics across both workloads.

Universal Optimization Patterns:

Reduced Precision Dominance: FP16/BF16 appears in all top-performing configurations across both phases, validating Tensor Core utilization as critical for efficiency [7]. This represents a universal optimization independent of workload type.

Pareto Optimality: Both phases discover distinct performance-optimal and energy-optimal regimes. Performance configurations prioritize maximum throughput/accuracy at moderate-high power (600-700W),

TABLE III
CROSS-PHASE EFFICIENCY COMPARISON

Metric	Phase 1	Phase 2
Primary Metric	%/Wh	tokens/Wh
Configurations Tested	113	125
Success Rate	$> 95\%$	65.6%
Best Efficiency Gain	$1.65\times$	$4.07\times$
Max Efficiency Gain	$2.68\times$	$8.09\times$
Throughput Multiplier	$1.50\times$	$3.47\times$
Power Range	400-700W	300-600W

while energy configurations achieve 99.6-99.7% of peak performance at substantially reduced power (300-400W), demonstrating NSGA-II's effectiveness for multi-objective GPU optimization [2].

Compounded Improvements: High efficiency multipliers result from addressing multiple bottlenecks simultaneously (precision + kernels/batch + power + memory/sequence), not single-parameter optimization. Best configurations optimize 3-4 parameters jointly.

Power-Performance Saturation: Both phases exhibit diminishing returns above 500W. Energy configurations exploit this saturation, achieving 99.6-99.7% of peak throughput at 43-57% lower power.

Target Achievement Validation:

The proposed research targeted 15-20% efficiency improvements. Achieved results substantially exceed this target:

- Phase 1: $1.65\text{-}2.68\times$ efficiency gains
- Phase 2: $4.07\text{-}8.09\times$ efficiency gains

V. CONCLUSION

This work presents an Adaptive Evolutionary GPU Optimization Framework (AEGOF) that addresses the critical problem of suboptimal GPU resource utilization in deep

learning deployments. By combining NSGA-II multi-objective evolutionary algorithm with domain-informed safety heuristics, AEGOF successfully optimizes 4-8 GPU hardware and software parameters while maintaining model accuracy and ensuring hardware safety.

Comprehensive experimental evaluation across 238 configurations demonstrates substantial improvements spanning two distinct Pareto frontiers. Phase 1 (ResNet50 training) discovers Best Performance configuration achieving 90.21% accuracy (8.46 point gain) with $1.65\times$ efficiency multiplier (22.10 vs. 13.37 %/Wh), and Energy Efficient configuration delivering $2.68\times$ efficiency gain (35.78 %/Wh) at 400W with minimal accuracy sacrifice. Phase 2 (Llama 2 inference) discovers Best Performance (257,274 tokens/Wh, $4.07\times$ multiplier @ 600W) and Energy Efficient (512,256 tokens/Wh, $8.09\times$ multiplier @ 300W), with the latter achieving 99.6% of maximum throughput at 50% power consumption.

The framework’s key innovations include parameter-type-specific mutation operators for mixed discrete-continuous spaces, proactive constraint handling preventing all thermal violations across 238 evaluations, hybrid initialization combining Latin hypercube sampling with GPU architecture heuristics, and multi-objective fitness enabling discovery of flexible deployment options. Mathematical validation using Watt-hour electricity units proves efficiency multipliers of $1.65\text{--}8.09\times$ across training and inference workloads, substantially exceeding the proposed 15-20% improvement target.

Universal optimization patterns emerge: reduced precision (FP16/BF16) with Tensor Core utilization proves essential for efficiency gains, Pareto-optimal solutions enable context-specific deployment across performance-critical and energy-constrained scenarios, compounded improvements from addressing multiple bottlenecks deliver superior results, and power-performance saturation above 500W enables energy configurations to achieve near-peak performance at substantially reduced power. At datacenter scale (1000 GPUs), Energy Efficient inference saves \$350,400 annually in electricity costs and avoids 2,102 metric tons of CO₂ emissions.

Future work should validate across diverse GPU generations, implement surrogate modeling to reduce evaluations, develop online adaptation mechanisms, extend to multi-GPU distributed optimization, and evaluate on broader workload coverage including Vision Transformers and diffusion models. The demonstrated effectiveness establishes a foundation for intelligent, sustainable AI infrastructure.

REFERENCES

- [1] F. Wang, W. Zhang, S. Lai, M. Hao, and Z. Wang, “Dynamic GPU Energy Optimization for Machine Learning Training Workloads,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2681–2694, Nov. 2022.
- [2] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.
- [3] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proc. 29th ACM Symp. Operating Systems Principles*, 2023, pp. 611–626.
- [4] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *Proc. 36th Conf. Neural Information Processing Systems*, 2022, pp. 16344–16359.
- [5] Z. Li, S. Yang, and S. Wang, “Toward Large-Scale Evolutionary Multi-tasking: A GPU-Based Paradigm,” *IEEE Trans. Evol. Comput.*, vol. 25, no. 5, pp. 973–985, Oct. 2021.
- [6] Y. Zhang, J. Zhang, Q. Lin, and C. Zhou, “A Comprehensive Review on Evolutionary Algorithms for Multi-objective Optimization,” in *Proc. IEEE Int. Conf. Software Quality, Reliability Security Companion*, Dec. 2023, pp. 456–463.
- [7] NVIDIA, “Train With Mixed Precision,” NVIDIA Developer Documentation, 2023.
- [8] NVIDIA, “NVIDIA H100 Tensor Core GPU Architecture,” NVIDIA Technical Whitepaper, 2022.
- [9] D. Patterson et al., “Carbon Emissions and Large Neural Network Training,” *arXiv preprint arXiv:2104.10350*, Apr. 2022.
- [10] H. Li, Z. Liang, and R. Cheng, “GPU-accelerated Evolutionary Many-objective Optimization Using Tensorized NSGA-III,” *IEEE Trans. Cybernetics*, vol. 54, no. 4, pp. 1589–1603, Apr. 2024.
- [11] A. Mirhoseini et al., “A Graph Placement Methodology for Fast Chip Design,” *Nature*, vol. 594, pp. 207–212, June 2021.
- [12] S. Udkar, M. Vaithianathan, M. Reddy, and V. Gupta, “Balancing Performance and Power Efficiency in Modern GPU Architecture,” in *Proc. IEEE Int. Conf. Emerging Intelligent Systems*, 2024, pp. 112–118.